

Bridge Day 10 INSTRUCTOR KEY

Cumulative Recap Before Repetition

Learning sequence: Predict - Trace - Explain - Test - Interpret

Friday, July 17, 2026 • 20 points • target 18 minutes

Name		UIN		Section	
------	--	-----	--	---------	--

Assessment boundary and evidence source

Contract: 07a04826c975

Cumulative foundations: input conversion, integer arithmetic, Boolean precedence, ordered decisions, f-strings

Scaffold level: independent cumulative decision program before while-loop instruction

Evidence source: the matching v5 workbook readiness/evidence pages and VIP activities 18.19, 18.21, 18.22.

Show intermediate state for trace credit. Program credit requires one coherent solution, a stated assumption when the prompt leaves one implicit, and at least one test whose expected result can distinguish correct from incorrect logic.

Item 1. Trace arithmetic and Boolean precedence. - 4 points

```
x = 17
y = 5
result = (x // y) ** 2 + x % y
ok = result >= 11 and not (x % 2 == 0)
print(result, ok)
```

Question: Show quotient, remainder, exponentiation, and both Boolean components before giving exact output.

Model response: 17 // 5 is 3 and 17 % 5 is 2, so result is 3 ** 2 + 2 = 11. result >= 11 is True and not (17 % 2 == 0) is not False, also True. Exact output: 11 True.

Item 2. Trace an ordered decision with a missing-work condition. - 4 points

```
score = 88
missing = True

if score >= 85 and not missing:
    message = "distinction"
elif score >= 70 or not missing:
    message = "pass"
else:
    message = "incomplete"

print(message)
```

Question: Name the first matching branch and explain why the higher label is not selected.

Model response: The first condition is False because not missing is False. The elif condition is True because score >= 70 is True, so exact output is pass. Branch order cannot make a branch execute when its complete condition is False.

Item 3. Repair a type mismatch before comparison. - 4 points

```
raw = "12"
limit = 10
print(raw >= limit)
```

Question: Name the actual error, explain its cause, and write a corrected comparison that evaluates numerically.

Model response: Python raises `TypeError` because `>=` cannot order a string and an integer. Correct with `value = int(raw)` followed by `print(value >= limit)`, which prints `True`.

Complete program - 8 points

- Read temperature as a float and calibrated as text, where exactly yes means calibrated.
- Print invalid when temperature is below -20 or above 60.
- Otherwise print ready only when calibrated is yes and temperature is from 15 through 35 inclusive; print hold for every other valid case.
- Use one if/elif/else chain and print exactly one word.

Program workspace

```
Model program
temperature = float(input())
calibrated = input()

if temperature < -20 or temperature > 60:
    print("invalid")
elif calibrated == "yes" and 15 <= temperature <= 35:
    print("ready")
else:
    print("hold")
```

Test input, expected result, and why this test matters

Reasoning and test evidence The invalid range is checked first. The ready branch then combines calibration with the inclusive operating range; the final else collects every valid but not-ready case. Protective tests include -20/yes -> hold, 15/yes -> ready, 35/yes -> ready, 36/yes -> hold, and 61/yes -> invalid.

Scoring guidance: 2 input/conversion, 2 invalid boundary, 2 combined ready condition, 1 exclusive branch structure, 1 exact output/tests.